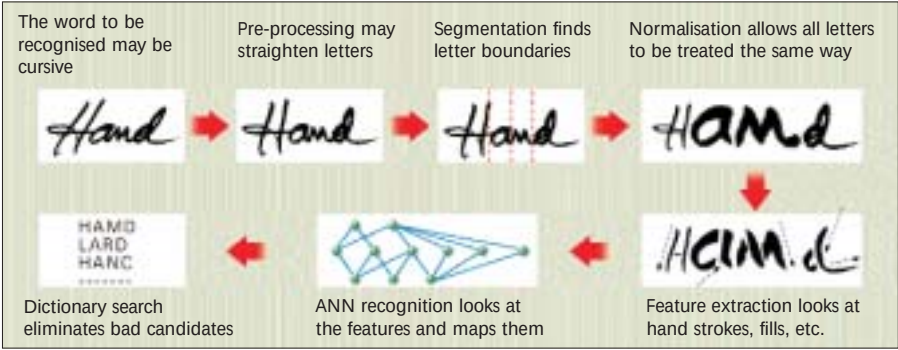




Typical stages in handwriting recognition using a neural net

give a uniform size to the text, straighten slanted lines in cursive writing, etc.

The most difficult part of the process is segmentation, i.e., breaking up the image into meaningful components, often single letters. Letter boundaries are hinted at by features such as changes in thickness, a change in slant, a small gap, etc. Using these, the segmentation process tries to isolate the letters in a word. The isolated letters may later be analysed according to their overall structure, or by individual features.

Segmentation is helped if the input is online—that is, if writing and recognition are being done simultaneously—because the system then has additional parameters to work on such as changes in pressure, small time gaps and so on.

After the letters have been demarcated, each letter or character is recognised by the ANN. The features that the system looks at

are essentially the building blocks of written text—vertical and horizontal strokes, curves, circles, etc. Contextualization comes in here: isolated characters are usually not fed to the ANN—it's usually small groups. This improves accuracy. For example, when writing the word 'her', say the 'h' looks more like a 'b'; the word will still be recognised correctly as 'her', because 'ber' isn't a word. After recognition, dictionary-based correction completes the process: if a word that's not a valid dictionary word turns up, it's replaced by the closest match. Several HR systems can learn, and thus get personalised to one's writing peculiarities. This leads to better accuracy.

Where theory ends and problems begin

The applications of speech recognition extend far beyond dictation software. As

we move away from the desktop and towards 'computers everywhere', we also move towards 'speech recognition everywhere'. However, there is no one-size-fits-all SR application: the demands of different areas are different.

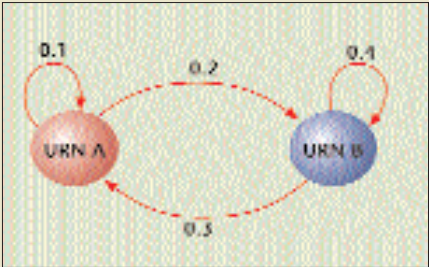
Consider voice recognition in a car. Hands-free mobile phone use is one thing, but what about voice-controlled, automated steering and guidance? Here, accuracy is of paramount importance, and there is very little space for 'ums' and 'ers'. The speaker must conform to speech-quality standards, and the software must be as accurate as possible. Fortunately, in such applications, the vocabulary the system needs to have is limited, making the task somewhat easier.

Background noise is a common problem with SR, and will remain so—even when partially remedied by noise-cancelling microphones. Consider the application of SR in labs, or in classrooms, where researchers take down notes while working with equipment; or where a lecturer's speech is automatically transcribed and presented to students after the lecture. Elimination of background noise is essential to good reproduction in these scenarios.

An interesting application is where software trains students to speak a language properly, with the correct accent and nuances of pronunciation. This poses a unique problem—a trade-off between how much the software should

Hidden Markov Models

HMMs are used to model what might be happening in a system, when there are both known and unknown parameters. In speech, the known parameters are the sounds that are heard, and the unknown parameters are what the speaker actually said. We need to try and guess the latter from the former. A simple way of understanding HMMs is via the urns and balls model. Think of two urns, each



Probabilities of the genie jumping between, or staying at, urns A and B

containing both red and blue balls. A genie keeps jumping back and forth between them. Each time he stops at an urn, he picks out a ball, and calls out what colour it is. We can hear him say the colour, but we don't know what urn he's calling out from. So, the urns are the unknown parameters, and the colour of the ball is the known parameter.

Say the probability of the genie jumping, or transitioning, between urns A and B, is as below:

This means that the probability of the

URN	A	B
Probability that a ball is red	0.3	0.6
Probability that a ball is blue	0.7	0.4

genie staying at urn A is 0.1, of going to B from A is 0.2, and so on. Assume that the initial probability of the genie being at A is 0.4, and of being at B is 0.6.

Now, suppose that the probabilities of red and blue balls being in urns A and B respectively is as follows:

Say we've heard the genie call out, in succession, "red", "blue" and "red". What sequences of moves by the genie could have produced this utterance, and with what probabilities? These can be calculated.

The probability that the genie was at urn A all three times, is $0.4 \times .3 + .1 \times .7 + .1 \times .3 = 0.22$.

This means that the sequence was 'AAA' with a 22 per cent probability. The probability that he jumped from B to A and then back to B, is:

$0.6 \times .6 + .3 \times .7 + .2 \times .6 = 0.69$.

This means that the sequence was 'BAB' with a probability of 69 per cent, and so on. Thus, given a sequence we can estimate, using an HMM, the underlying sequence that produced it.